

MLIB Co-Development 2016

Nicholas Burgess



MHI

Nicholas.Burgess@uk.mizuho-sc.com

22nd November 2016

Global Quant Strategy

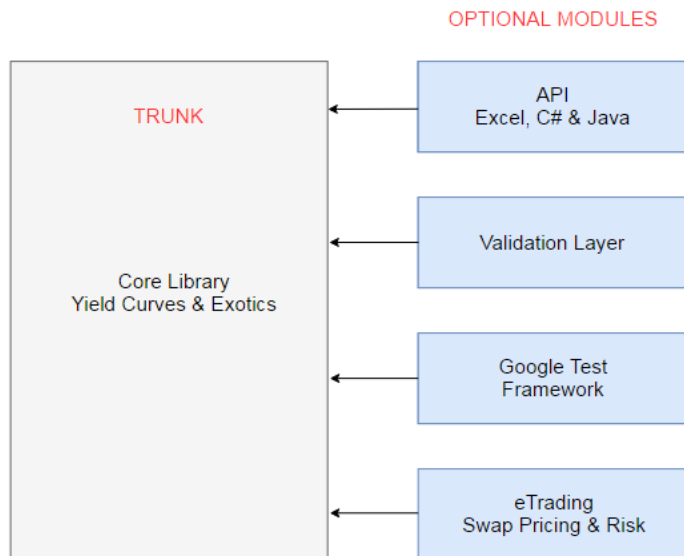
- MLIB is being co-developed by Tokyo and London Quants
- Together we address the needs of a global business
- Potentially MCMC New York will also co-develop in the near future

MLIB Co-Development 2016

- Modified library configuration to compile and initialize for global use
- International code made independent & modular for scalability
- Yield curve bug fixing & enhancements
- Extended product & risk offering
- Added test framework
- Added C# and Java API

- 1 Co-Development Strategy
 - The Importance of Updating the Trunk
- 2 Folder Structure
- 3 MLIB Configuration
- 4 Extensions to Core Analytics
 - Yield Curve Enhancements
 - Extended Swap Functionality
- 5 Optional Project Extensions
 - Etrading Project
 - Google Test Project
 - Validation_API Project
 - MLIB_Client_API Project
- 6 Appendix
 - Code Structure

Library Structure



Project Structure

- Core: Yield Curve & Exotics
- Optional Projects: Modular & Independent

Solution Files

- **irsv_dev.sln:**
Original native solution for Tokyo use, excludes optional modules
- **irsv_dev_global.sln:**
For global use and includes optional modules
- **irsv_dev_everything.sln:**
Includes everything, both optional modules & Tokyo server projects

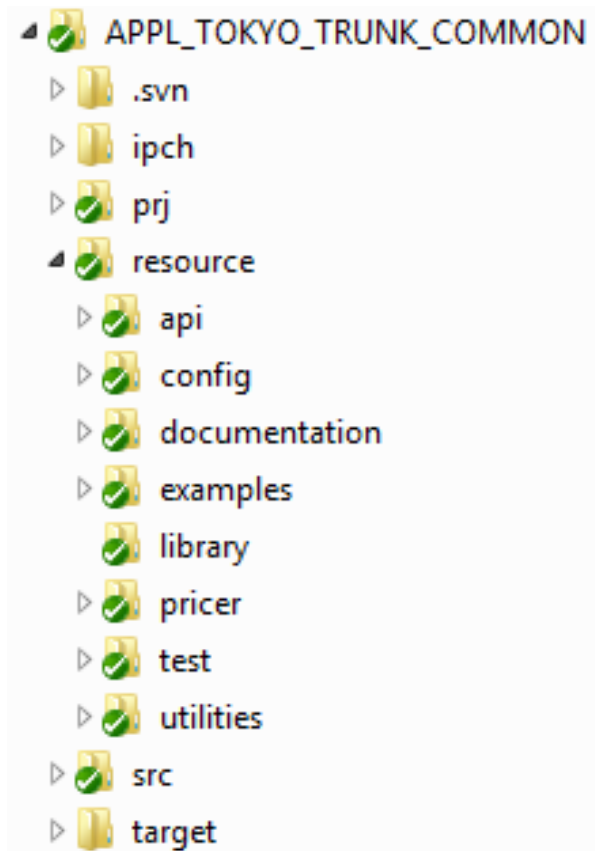
Updating the Trunk

- The trunk is the master copy of the library
- All copies of the library are taken from the trunk
- Bug fixes not in the trunk leave the master copy broken
- When the trunk is broken all branches will be broken
- Enhancements are only available to the team after merging
- Branches not merged for long periods of time lead to code divergence

Summary

- Merging is required to share bug fixes and enhancements
- Merging leads to teamwork and collaboration
- Teamwork is not possible without updating / merging branches back to the trunk

Folder Structure



Main Folders

prj: Project Files

src: Source Code

target: Object Files & Binaries

resource Folder

- **api**: C# and Java API source code
- **config**: calendars and property config files
- **documentation**: docs go here
- **examples**: excel example templates
- **library**: optional external library folder
- **pricer**: excel pricing workbooks
- **test**: google test case folder
- **utilities**: batch files, Excel toolbar etc

Environment Variables

Two environment variables have been introduced to allow MLIB to be used and configured for global use. Previously configuration was hard-coded, which prevented co-development.

"MLIB"

This environment variable sets where the library is located. Configuration and property files are relative to this root folder.

"MLIB_EXTERNAL_LIB_PATH"

This sets where the external libraries such as Boost are stored. Using this environment variable means we only need to store external libraries once, rather than once per branch or check-out.

Yield Curve Enhancements :-

- Improved curve speed to milliseconds
- Added curve tension & monotone convex interpolation
- Added Fed Fund, ECB swaps & Libor-OIS basis to OIS curve
- Added forward starting swaps to STD swap curve
- Extended convexity adjustment methods and added overrides
- Extended for multiple fx forward curves to improve CSA curve speed

Code Structure

Curve enhancements have been made independently to simplify the code merging process and can be used independently or merged back to the library core if required. See Appendix 1: Code Structure.

New Swap Functions

- `mirAssetSwapSpread`: par-par asset swap spread
- `mirCompound3`: OIS effective compound rate
- `mirGetParRate4`: swap par rate calculation
- `mirSwapSchedule`: display a swap schedule
- `mirSwapStubRate`: swap stub rate
- `mirSwapPV`: swap PV
- `mirSwapPV01`: swap PV01 or annuity
- `mirSwapDV01`: swap DV01
- `mirSwapDeltaLadder`: bucketed swap delta
- `mirOISParRate`: OIS par rate
- `mirOISPV`: OIS PV

Purpose: Simplicity, Transparency, Configurability, Serialization

Features

① Date Generation

Generation of schedules, year fractions and dates

② Extended Curve Building with CSA Support

③ Simplified Discount Factor & Forward Calculations

④ Swap Pricing

OIS, IRS, Tenor Basis, Xccy, Asset Swaps, Amortizers etc.

⑤ Detailed Cashflow, Price & Risk Breakdowns

⑥ Object & Serialization Support

Data Grids, Fixing Tables, Fee Schedules, Swaps & Curve Serialization

⑦ Added a test framework, logging, record and replay features

⑧ Added automation build, test and deployment scripts

Test Fixtures & Google Test Requirements

To compliment Google Test "[test fixtures](#)" or classes are used for complex, large and model tests.

Models and yield curves can be recorded, loaded, preloaded, updated and re-based, however the "[validation_api](#)" project is required for this.

Features

- 200+ Test Cases
- Test Fixture Classes for More Complex Tests
- Test Recording & Playback
- Automated Test Scripts

Overview of Validation_API Project

The validation project is a thin project layer that sits on top of the analytics. The validation layer provides a single common entry point to the analytics. This ensures that all client applications and APIs such as Excel, C#, Java consistently return the same results. We consistently perform all input validation and application crash checks in this project.

Features

- Validation of User Inputs
- Single Entry Point into Analytics for all Client Apps
- Test Recording & Playback
- Excel and other API Application Crash Checks & Controls

Overview

This project is an interface generator used to generate binaries to allow external libraries written in other languages to connect to MLIB.

Features

- **Swigwin**

The project uses the Swigwin open-source package

- **C# & Java**

Currently this project generates API code to support C# and Java

- **26 Languages**

The framework supports up to 26 different coding languages

- **Future Work**

R and Python API support is likely to be added soon

Yield Curve Enhancements

Yield Curve enhancements have been made independently to simplify the code merging process and can be used independently or merged back to the library core if required. Independent copies of core analytics have been made for this purpose, see below.

1. Copying from ir, irserver projects to etrading project:

- [MASDEGenerator](#) - Calibrator
- [MASDEGeneratorIR](#) - CalibratorIR
- [MASDEGeneratorIRVanilla](#) - CalibratorIRVanilla
- [MAExcelAddinWrap](#) - YieldCurveFactory
- [MFirYieldCurvePro](#) - YieldCurvePro
- [MFirYieldGenerator](#) - YieldGenerator

2. Copying from vanilla project to etrading project:

- [InitializeMLibVanilla](#) - InitializeMLibETrading
- [LibSetUp](#) - LibSetUpETrading
- [MAExcelAddinConfig](#) - ExcelAddinConfig

3. Copying from VANILLA_ADDIN project to ETRADING_ADDIN project:

- [XIIIInterf](#) - vanillaAddinXL
- [XIIIPlusTips](#) - vanillaAddinXLUtils

The End